

Bildfilter in Python

Projekt mit Stempel-Wanderung

Lars Rafeldt

*MNG, Mathematisch-Naturwissenschaftliches Gymnasium Rämibühl
Fachschaft Informatik*

E-mail: lars.rafeldt@mng.ch

Klasse 1f, Frühlingssemester 2026



mng | k+s rämibühl

**Mathematisch-Naturwissenschaftliches Gymnasium
Kunst und Sport Gymnasium**

1 0010 1011 0011 0111 0011 1000
0 1010 1111 0101 1101 0000 0011
0 0010 0011 0010 0011 1011 0001
0 0000 0000 0001 0010 1101 1110
1 1100 1010 0110 0110 0000 1111

Projektbeschreibung

Lernziele:

- Du verstehst, wie ein Computer ein Bild speichert.
- Du kannst eigene Bildfilter als Python-Funktionen schreiben und an echten Fotos ausprobieren.
- Du gehst Schritt für Schritt von einfachen Pixel-Änderungen bis zu Filtern, die mehrere Nachbarpixel zusammenrechnen.

Jedes Mal, wenn du auf Instagram einen Filter anwendest oder in der Galerie-App ein Foto heller machst, läuft im Hintergrund ein Bildbearbeitungs-Algorithmus. In diesem Projekt programmierst du solche Filter selbst und siehst sofort, was dein Code mit echten Bildern macht.

So funktioniert das Projekt

- (a) Du arbeitest an deinem eigenen Python-Programm. Du schreibst Filter als Funktionen.
- (b) Auf einer Website führst du ein *Stempelheft*. Für jede Aufgabe, die du schaffst, bekommst du einen Stempel.
- (c) Mit jedem Stempel wird ein Stück eines paint-by-numbers-Bildes auf deinem Heft sichtbar. Wer alle Stempel hat, sieht das fertige Bild.

Ein **Bildfilter** ist ein Algorithmus, der ein Bild als Eingabe nimmt und ein neues Bild zurückgibt. Dabei wird jeder Pixel nach einer bestimmten Regel verändert.

Spielregeln

- Du arbeitest mit **verschachtelten Listen**, wie du sie aus dem Algorithmen-Skript kennst:

```
bild[zeile][spalte]  # liefert einen Pixel (R, G, B)
```

- Zum Laden und Speichern von Bilddateien verwendest du die Bibliothek `Pillow`. Den genauen Code dafür findest du in Schritt 0 des Lernpfads.
- Alles andere (jede Filterlogik) schreibst du selbst.

Beispiel

Was ist ein Pixel?

Ein Pixel wird in Python als Tupel aus drei Zahlen gespeichert, jeweils zwischen 0 und 255:

```
schwarz = (0, 0, 0)
weiss    = (255, 255, 255)
rot      = (255, 0, 0)
gruen    = (0, 255, 0)
blau     = (0, 0, 255)
gelb     = (255, 255, 0)    # Rot + Gruen
violett  = (128, 0, 128)
```

Auf einen einzelnen Pixel greifst du so zu:

```
pixel = bild[10][20]    # Pixel in Zeile 10, Spalte 20
r, g, b = pixel          # in seine drei Werte zerlegen
```

Abgabe

Du gibst am Ende drei Sachen ab:

- (a) Deine **Python-Datei(en)** mit allen Filtern.
- (b) Drei **Vorher/Nachher-Bildpaare**: ein Originalfoto und daneben jeweils das Ergebnis eines deiner Filter.
- (c) Einen **Bericht** (2 bis 3 A4-Seiten): Was hast du gemacht? Welcher Filter hat dich überrascht? Wo bist du steckengeblieben?

Die Abgabe läuft über die normale Plattform der Klasse. Wie genau, besprechen wir gemeinsam.



Überblick:

- Du kannst insgesamt **120 Punkte** erreichen, verteilt auf vier Bereiche.
- Den grössten Teil holst du dir mit den **Pflichtfiltern**. Daneben zählen dein **eigener Filter**, deine **Codequalität** und eine ordentliche **Abgabe**.
- Die folgende Grafik zeigt, wieviele Punkte in jedem Bereich zu holen sind.

A. Pflichtfilter 50 Punkte	B. Eigen 20 Pkt.	C. Code 15 Pkt.	D. Abgabe 35 Punkte
--------------------------------------	----------------------------	---------------------------	-------------------------------

A. Pflichtfilter (50 Punkte)

Für jeden dieser Filter gibt es Punkte, wenn er korrekt funktioniert und ein sichtbares Ergebnis liefert. Jeder Filter ist eine eigene Funktion.

Filter	Was er tut	Pkt.
Kein Rot	Setzt den Rot-Wert aller Pixel auf 0.	5
Invertieren	Aus jedem Wert wird $255 - \text{Wert}$.	7
Graustufen	Pro Pixel: $\text{grau} = (R + G + B) // 3$, neuer Pixel (grau, grau, grau).	7
Helligkeit	Addiert eine Zahl h auf jeden Farbkanal, klemmt die Werte auf $[0, 255]$.	8
Schwelldwert	Pixel über einem Schwellwert t werden weiss, alle anderen schwarz.	7
Box-Blur	Pro Pixel: Durchschnitt aus dem Pixel selbst und seinen 8 Nachbarn (3×3 -Fenster).	10
Spiegeln	Spiegelt das Bild horizontal (linke und rechte Seite tauschen).	6

B. Eigener Filter (20 Punkte)

Erfinde und implementiere einen *eigenen* Filter, oder wähle einen aus den Vorschlägen am Ende des Lernpfads.

Kriterium	Anforderung	Pkt.
Funktion arbeitet korrekt	Der Filter produziert sichtbar das, was er soll.	10
Eigenständige Idee	Du kannst in zwei Sätzen erklären, was dein Filter tut und warum.	5
Beispielbild	Du zeigst ein Vorher/Nachher-Bild, auf dem der Effekt klar zu sehen ist.	5

C. Codequalität (15 Punkte)

Kriterium	Anforderung	Pkt.
Aussagekräftige Namen	Variablen heissen wie das, was sie sind: <code>zeile</code> , <code>spalte</code> , <code>neuer_pixel</code> , nicht <code>a</code> , <code>b</code> , <code>tmp</code> .	5
Eine Funktion pro Filter	Jeder Filter ist eine eigene Funktion, nicht alles in einem grossen Block.	5
Clamping korrekt	Bei <code>helligkeit</code> bleiben die Werte zwischen 0 und 255.	3
Rand bewusst behandelt	Beim Box-Blur weisst du, was du am Bildrand machst (z.B. einfach weglassen).	2

D. Abgabe (35 Punkte)

Kriterium	Anforderung	Pkt.
Code abgegeben	Alle Python-Dateien sind läuffähig und an die richtige Stelle hochgeladen.	5
Drei Beispielbild-Paare	Drei Original/Filter-Paare als Bilddateien.	5
Bericht	2 bis 3 A4-Seiten: Was hast du gemacht? Was war schwierig? Was hat dich überrascht?	25



Wegweiser:

- Jeder Schritt baut auf dem vorhergehenden auf.
 - Jeder Schritt enthält: was du tun sollst, ein kleines Code-Beispiel und welchen Stempel du dabei sammelst.
 - Wenn du eine Station geschafft hast, zeig der Lehrperson dein Resultat, und sie setzt den Stempel direkt in dein Heft.
-

Schritt 0: Bild laden

Bevor du Filter schreiben kannst, musst du ein Bild laden. Dazu verwenden wir die Python-Bibliothek `Pillow`. Lege ein Foto in denselben Ordner wie dein Python-Programm und lade es mit folgendem Code:

```
from PIL import Image

img = Image.open("foto.jpg").convert("RGB")
breite, hoehe = img.size

bild = []
for y in range(hoehe):
    zeile = []
    for x in range(breite):
        zeile.append(img.getpixel((x, y)))
    bild.append(zeile)
```

```
print("Hoehe:", hoehe, " Breite:", breite)
```

Die ersten Zeilen öffnen das Bild und holen seine Grösse. Danach bauen die zwei verschachtelten Schleifen das Bild als verschachtelte Liste von Pixel-Tupeln (R, G, B) auf, mit der du im Rest des Projekts arbeitest. Zum Speichern eines Bildes verwendest du am Ende:

```
ergebnis_img = Image.new("RGB", (breite, hoehe))
for y in range(hoehe):
    for x in range(breite):
        ergebnis_img.putpixel((x, y), bild[y][x])
ergebnis_img.save("ergebnis.png")
```

Stempel 1 *Erstes Bild*

Schritt 1: Erste Pixel verändern

Bevor wir richtige Filter schreiben, probieren wir aus, was passiert, wenn wir einzelne Farbkanäle verändern.

Aufwärmer: Rot weg

Setze den Rot-Wert aller Pixel auf 0. Was passiert?

Funktion `kein_rot(bild)`:
Erstelle ein leeres `neues_bild`.
Fuer jede Zeile **des** Bildes:
 Erstelle eine **leere** `neue_zeile`.
 Fuer jeden Pixel (r, g, b) **der** Zeile:
 Haenge $(0, g, b)$ **an** `neue_zeile` **an**.
 Haenge `neue_zeile` **an** `neues_bild` **an**.
Gib `neues_bild` **zurueck**.

Stempel 2 *Kein Rot*

Filter 1: Invertieren

Beim Invertieren wird jeder Farbwert “umgedreht”: aus 0 wird 255, aus 255 wird 0, aus 100 wird 155.

Definition

Invertieren

Für jeden Pixel (R, G, B) :

$$(R', G', B') = (255 - R, 255 - G, 255 - B)$$

Funktion `invertieren(bild)`:
Erstelle ein leeres `neues_bild`.
Fuer jede Zeile **des** Bildes:
 Erstelle eine **leere** `neue_zeile`.

```
Fuer jeden Pixel (r, g, b) der Zeile:  
    Haenge (255-r, 255-g, 255-b) an neue_zeile an.  
    Haenge neue_zeile an neues_bild an.  
Gib neues_bild zurueck.
```

Stempel 3 *Spiegelwelt*

Schritt 2: Punktoperationen

Filter, bei denen jeder neue Pixel nur vom *gleichen* Pixel im Originalbild abhängt, heissen **Punktoperationen**.

Filter 2: Graustufen

In einem Graustufenbild sind alle drei Farbkanaäle gleich. Der einfachste Weg: den Durchschnitt der drei Werte nehmen.

Definition	Graustufen
Für jeden Pixel (R, G, B) :	
$\text{grau} = \frac{R + G + B}{3}, \quad \text{neuer Pixel} = (\text{grau}, \text{grau}, \text{grau})$	

```
Funktion graustufen(bild):  
    Erstelle ein leeres neues_bild.  
    Fuer jede Zeile des Bildes:  
        Erstelle eine leere neue_zeile.  
        Fuer jeden Pixel (r, g, b) der Zeile:  
            Setze grau = (r + g + b) ganzzahlig durch 3.  
            Haenge (grau, grau, grau) an neue_zeile an.  
        Haenge neue_zeile an neues_bild an.  
    Gib neues_bild zurueck.
```

Stempel 4 *Graustufen*

Filter 3: Helligkeit

Um ein Bild heller zu machen, addieren wir auf jeden Farbwert eine Zahl h . Achtung: die Werte müssen zwischen 0 und 255 bleiben! Dafür schreiben wir eine kleine Hilfsfunktion `clamp`.

```
Funktion clamp(wert):  
    Wenn wert kleiner als 0: gib 0 zurueck.  
    Wenn wert groesser als 255: gib 255 zurueck.  
    Sonst: gib wert zurueck.  
  
Funktion helligkeit(bild, h):  
    Erstelle ein leeres neues_bild.  
    Fuer jede Zeile des Bildes:  
        Erstelle eine leere neue_zeile.
```

```

Fuer jeden Pixel (r, g, b) der Zeile:
    Haenge (clamp(r+h), clamp(g+h), clamp(b+h)) an neue_zeile
    an.
    Haenge neue_zeile an neues_bild an.
Gib neues_bild zurueck.

```

Stempel 5 *Heller, dunkler*

Aufgabe 1

Helligkeit von Hand

Was kommt heraus, wenn du auf einen Pixel (200, 100, 50) die Helligkeit

- (a) $h = 50$
- (b) $h = -150$
- (c) $h = 100$ (ohne Clamping)

anwendest?

Schritt 3: Schwellwert

Beim Schwellwert wird jeder Pixel *entweder* ganz schwarz *oder* ganz weiss, je nachdem, ob seine Helligkeit über oder unter einem Schwellwert t liegt.

Definition	Schwellwert
Für jeden Pixel berechne zuerst den Grauwert g . Dann:	
$\text{neuer Pixel} = \begin{cases} (255, 255, 255) & \text{falls } g \geq t \\ (0, 0, 0) & \text{sonst} \end{cases}$	

```

Funktion schwellwert(bild, t):
    Erstelle ein leeres neues_bild.
    Fuer jede Zeile des Bildes:
        Erstelle eine leere neue_zeile.
        Fuer jeden Pixel (r, g, b) der Zeile:
            Setze grau = (r + g + b) ganzzahlig durch 3.
            Wenn grau groesser oder gleich t:
                Haenge (255, 255, 255) an neue_zeile an.
            Sonst:
                Haenge (0, 0, 0) an neue_zeile an.
        Haenge neue_zeile an neues_bild an.
    Gib neues_bild zurueck.

```

Stempel 6 *Schwarz-Weiss*

Schritt 4: Nachbarpixel und Box-Blur

Die bisherigen Filter haben jeden Pixel *einzel*n bearbeitet. Spannender wird es, wenn wir auch die **Nachbarpixel** mitnehmen.

Definition

Box-Blur (3×3)

Für jeden Pixel: schau dir den Pixel selbst und seine 8 direkten Nachbarn an (das 3×3 -Fenster). Berechne pro Farbkanal den **Durchschnitt** aller 9 Werte. Das ist dein neuer Pixel.

NW	N	NO
W	Pixel	O
SW	S	SO

Funktion box_blur(bild):

Erstelle ein leeres neues_bild.

Für jede Zeile **von 1 bis** hoehe-2:

Erstelle eine **leere** neue_zeile.

Für jede Spalte **von 1 bis** breite-2:

Setze summe_r, summe_g, summe_b auf 0.

Für dz **in** {-1, 0, 1}:

Für ds **in** {-1, 0, 1}:

Lies Pixel (r, g, b) **an** (zeile+dz, spalte+ds).

summe_r = summe_r + r

summe_g = summe_g + g

summe_b = summe_b + b

Haenge (summe_r//9, summe_g//9, summe_b//9) **an** neue_zeile

an.

Haenge neue_zeile **an** neues_bild **an**.

Gib neues_bild **zurueck**.

Beachte: wir beginnen bei Zeile/Spalte 1 und hören eine Zeile/Spalte vor dem Ende auf. So haben wir immer ein vollständiges 3×3 -Fenster. Die äussersten Pixel fallen dabei weg. Das ist eine bewusste Entscheidung und für unsere Zwecke völlig in Ordnung.

Stempel 7 *Nachbarn mischen*

Aufgabe 2

Mehrfach blurren

Was passiert, wenn du den Box-Blur *mehrfach hintereinander* auf dasselbe Bild anwendest? Probiere $1\times$, $5\times$ und $20\times$ aus.

Schritt 5: Bild umordnen, Spiegeln

Bisher haben wir Pixel *verändert*. Jetzt ordnen wir sie um. Beim horizontalen Spiegeln tauschen wir die linke und die rechte Seite.

```

Funktion spiegeln(bild):
    Erstelle ein leeres neues_bild.
    Fuer jede Zeile des Bildes:
        Erstelle eine leere neue_zeile.
        Fuer jede Spalte von 0 bis breite-1:
            Haenge bild[zeile][breite-1-spalte] an neue_zeile an.
        Haenge neue_zeile an neues_bild an.
    Gib neues_bild zurueck.

```

Stempel 8 *Spiegelei*

Schritt 6: Dein eigener Filter

Jetzt bist du dran! Erfinde *einen* eigenen Filter. Hier ein paar Ideen (du darfst auch etwas ganz Eigenes machen):

- **Vertikal spiegeln:** obere und untere Hälfte tauschen.
- **Posterisierung:** runde jeden Kanal auf Vielfache von 64 (Werte werden dann nur noch 0, 64, 128, 192).
- **Verpixelung:** ersetze jeden $k \times k$ -Block durch die Durchschnittsfarbe dieses Blocks.
- **Einfache Kantenerkennung:** für jeden Pixel den Unterschied zum rechten Nachbarn berechnen.
- **Nur ein Kanal:** alle Farben ausser Rot, Grün oder Blau auf 0 setzen.

Stempel 9 *Eigener Filter*

Schritt 7: Abgabe

Bereite die Abgabe vor:

- Prüfe, dass alle Pflichtfilter laufen.
- Suche drei Originalbilder aus und wende je einen Filter darauf an. Das ergibt drei *Vorher/Nachher*-Paare.
- Schreibe einen Bericht von 2 bis 3 A4-Seiten. Was war einfach? Was war schwierig? Welcher Filter hat dich überrascht?
- Gib Code, Bilder und Bericht ab.

Stempel 10 *Auf dem Gipfel*



Worum es geht:

- Auf japanischen Wanderwegen gibt es an jeder Station einen Stempel, den du in dein Heft drückst. Wer alle Stempel hat, hat den Berg geschafft.
 - Wir machen das mit Code: jede gemeisterte Station gibt **einen Stempel**. Mit jedem Stempel wird ein Teil deines *paint-by-numbers*-Bildes sichtbar.
 - Es gibt insgesamt **10 Stationen**, vom ersten geladenen Pixel bis zur fertigen Abgabe.
-

So funktioniert es

- (a) Zu Beginn des Projekts bekommst du eine **Login-Karte** mit deinem Benutzernamen und einem Passwort.
 - (b) Damit öffnest du die Website *Bildfilter Stempel-Wanderung* im Browser und meldest dich an.
 - (c) Wenn du eine Station geschafft hast, zeigst du dein Resultat der Lehrperson.
 - (d) Die Lehrperson setzt dir den Stempel direkt in dein Heft. Die nächsten zwei Farben auf deinem Bild werden sofort sichtbar.
-

Auf deiner Login-Karte stehen:

- dein **Benutzername** (in der Regel dein Vorname, klein geschrieben),
- dein **Passwort** (sechs Zeichen),
- die **Adresse** der Website.

Bewahre die Karte gut auf. Wenn du sie verlierst, sag der Lehrperson Bescheid, dann gibt es ein neues Passwort.

Der Trail



Stationen im Detail

#	Stempel	Anforderung	Lernpfad
1	Erstes Bild	Bild mit <code>lade_bild</code> in eine Liste laden, Höhe und Breite ausgeben.	Schritt 0
2	Kein Rot	Rot-Wert aller Pixel auf 0 setzen und speichern.	Schritt 1
3	Spiegelwelt	<code>invertieren(bild)</code> : jeder Wert wird zu $255 - \text{Wert}$.	Schritt 1
4	Graustufen	<code>graustufen(bild)</code> : Mittelwert $\frac{R+G+B}{3}$.	Schritt 2
5	Heller, dunkler	<code>helligkeit(bild, h)</code> mit Clamping auf $[0, 255]$.	Schritt 2
6	Schwarz-Weiss	<code>schwellwert(bild, t)</code> : Pixel über t werden weiss.	Schritt 3
7	Nachbarn mischen	<code>box_blur(bild)</code> : 3×3 -Nachbarschaft mitteilen.	Schritt 4
8	Spiegelei	<code>spiegeln(bild)</code> : horizontal spiegeln.	Schritt 5
9	Eigener Filter	Ein selbst erfundener Filter, der sichtbar funktioniert.	Schritt 6
10	Auf dem Gipfel	Code, drei Vorher/Nachher-Bildpaare und Bericht (2 bis 3 A4-Seiten) abgeben.	Schritt 7

Das Bild

Dein Stempelheft zeigt eine *paint-by-numbers*-Vorlage des Berges Fuji mit einer japanischen Pagode. Am Anfang ist nur die Vorlage mit den Zahlen zu sehen. Mit jedem Stempel wird ein Stück der Vorlage durch das *fertige Bild* ersetzt, das, was du am Ende der Wanderung in voller Farbe siehst.

Insgesamt ist das Bild in zehn Bereiche aufgeteilt, einer pro Stempel.

